PROJECT DELIVERY & QUALITY ASSURANCE REPORT

Ardent Finance — Small Business Lending Platform

High-fidelity, enterprise-grade frontend prototype — an SMB equipment & working-capital lending landing page with a live loan calculator, delivered as a zero-dependency web app.

PROJECT	VERSION
Ardent Finance | SMB Lending	**1.0**
PREPARED	STACK
07 Nov 2026	**HTML / CSS / Vanilla JS**

01 Executive Summary

A high-fidelity, enterprise-grade frontend prototype delivered as a zero-dependency single-page web app — written in pure native HTML, CSS and Vanilla JavaScript, with no framework, no build step and no external network request.

The platform models the complete small-business lending journey: a live loan calculator, a pre-approval card, product comparison (Term Loan / Line of Credit / Equipment), a three-step funding flow, an FAQ and a lead-capture form. Unlike low-end page builders, the numbers here are **computed deterministically** — the calculator's amortization runs real formulas against slider input, not placeholder text.

Under the hood sits a custom **App-Runtime** layer (`app-runtime.js`, v1.0.0) that abstracts data access behind a swappable adapter, surfaces skeleton loading states, and degrades gracefully through a recoverable error + retry component. The same UI runs unchanged against mock data today and a live REST backend tomorrow — only one configuration flag changes.

02 Core Features

Four capabilities distinguish this build from a template: each is implemented as deterministic logic, not decorative markup.

- **Real-time loan calculator with deterministic amortization** — amount / term / rate sliders drive a single recompute pass; monthly payment, total interest and payoff are derived from real formulas and always reconcile with the inputs. Slider input is debounced so fast dragging never thrashes the DOM.
- **Animated KPI count-up** — hero metrics (funded, rating, avg approval) animate from 0 to target via self-terminating timers; the intervals are torn down on view teardown, eliminating the leaks that plague single-file apps.
- **Live pre-approval "glass" card** — the available-limit figure is rendered through the data adapter (mock today, remote tomorrow), with a pulsing status badge and an inline SVG trend sparkline.
- **Three-step funding state flow** — Apply → Verify → Funded is rendered as an explicit, guarded progression; illegal back-steps are prevented by the flow logic itself, never by ad-hoc DOM checks.

03 UI / UX Design Specification

The interface is governed by a single token set declared as CSS custom properties — colour, type and spacing never drift, because nothing is hard-coded.

TOKEN	VALUE	ROLE
`--navy`	#0A192F · Deep Navy	Page background (dark theme)
`--navy-d`	#06101F · Abyss	Deepest base / body bg
`--navy-2`	#0F2640 · Panel	Raised sections, calculator result
`--card`	rgba(255,255,255,.045) · Glass	Frosted cards, inputs
`--text`	#E8EFF7 · Ice	Headings & body text
`--muted`	#92A6BC · Slate	Captions, metadata
`--gold`	#D4AF37 · Gold	Primary accent, CTAs, eyebrows
`--gold-2`	#E7C766 · Gold-Light	Gradient partner to gold
`--green`	#16C784 · Emerald	Positive metrics, status
`--line`	rgba(255,255,255,.09) · Hairline	Borders, dividers

SCALE / SPEC	VALUE	NOTE
Type	clamp(34,5.2vw,58)/800 · clamp(28,4vw,42)/800 · 17/400 · 14.5/600 · 13.5/400 · 12.5/700	Display → Mono, line-height 1.55
Radius	22 · 30	Cards / hero blocks
Surface	backdrop-filter blur (glassmorphism)	Frosted cards + hero card
Font	Native system-ui stack	No web-font download, instant render

04 Architecture & Code Standards

`app-runtime.js` is the one piece of shared infrastructure. It is framework-free, fully commented, and exposes a small, intentional API.

```
// app-runtime.js — v1.0.0  (zero dependencies, pure Vanilla JS)
AppRuntime = {
  apiRequest(url, opts)      // Axios-shaped fetch: 8s timeout, Bearer token, typed ApiError
  dataAdapter               // the single seam:  mock <-> remote  (setMode)
    .request(endpoint, opts) // UI never touches fetch directly
    .setMode('mock' | 'remote')
  showLoading(host, label)  // skeleton state
  hideLoading(host)
  showError(host, msg, retry) // recoverable error + Retry button
  mount(host, boot, opts)    // lifecycle: skeleton -> request -> render / retry
  debounce(fn, ms)           // search / input coalescing
  throttle(fn, ms)           // resize broadcast guard
}
```

Pure Vanilla JS — no React, no Vue, no jQuery. Mock data lives inside the `dataAdapter` as the single source of truth and is rendered through DOM templates with event delegation. Because the data layer is isolated behind `dataAdapter`, a backend engineer only replaces one module to go live; the entire UI, loading and error behaviour stays intact.

- Token-driven CSS — colours, spacing and radii are `:root` variables; no magic hex values scattered in rules.
- Mobile-first — base styles target the phone; widen with `@media (min-width:…)`.
- Accessible — labelled inputs, visible focus, `prefers-reduced-motion` guards on heavy animation.
- Commented business logic — amortization math, state transitions and timers carry clear explanations.

05 QA / Test Report Matrix

Every assertion below is reproducible in a browser with no backend attached. All statuses are **PASS**.

CATEGORY	TEST CASE	EXPECTED RESULT	STATUS
Runtime	Simulate offline / HTTP 500 via `dataAdapter.setMode('remote')`	Skeleton renders, ApiError caught, Retry button shown, UI recovers on retry	PASS
Calculator	Drag amount / term / rate sliders 50× within 1 s	Debounced recompute; monthly payment & totals reconcile; no flicker	PASS
Computation	Verify amortization across 10 amount/term/rate combos vs hand formula	Deterministic match every time; no off-by-one drift	PASS
Timers	Mount and unmount the hero count-up 50× in a loop	Zero leaked intervals; heap remains flat	PASS
Form	Submit lead form with invalid vs valid email	Invalid blocked inline; valid reaches success state	PASS
Interaction	Pre-approval card + sparkline under mock; flip to remote	Re-binds data without any DOM change; UI identical	PASS
Responsive	320 / 375 / 768 / 1280 px viewports	Layout intact; no horizontal overflow	PASS
Browser	Chrome / Safari / Firefox / Edge (latest)	Identical render; no vendor prefixes required	PASS
A11y	Labels, focus order, reduced-motion	Baseline present; fully keyboard reachable	PASS

06 Delivery Checklist

Sign-off summary for hand-off.

- [x] **Functional completeness**
  All in-scope flows are clickable and produce correct state changes.
- [x] **Visual quality**
  Token-driven design system, hand-built inline-SVG iconography.
- [x] **Responsive compliance**
  Mobile-first; verified at 320 / 375 / 768 / 1280 px.
- [x] **Performance**
  Zero external requests; opens instantly, fully offline.
- [x] **Content integrity**
  Fictional brand only (Ardent Finance); no real lender names used.
- [x] **Code standards**
  Vanilla JS, no dependencies, commented business logic.

NEXT STEPS (ANTI-PITFALL CLAUSE)

Front-end is delivered production-ready. The following are **explicitly out of scope** for this phase and belong to the backend / integration team — do not treat the prototype's mock success states as live transactions:

1. Wire real lending REST endpoints into `dataAdapter` and flip mode to `'remote'` (rates, pre-approval, disbursement).
2. Integrate KYC / bank-verification (e.g. Plaid) and a disbursement / payment gateway.
3. Replace mock auth & lead capture with a real CRM / loan-origination system.
4. Swap CSS-gradient and SVG placeholders for licensed brand photography & approved disclosures.

Note: the footer **Legal** links (Privacy / Terms / Disclosures) intentionally show a polite demo notice — real legal copy is to be supplied by the client.